

TITLE AND DATE

Document name: Project#5 EEPROM Semaphore Lab Report

Document reference: <Demory, Nathan>.cmpe311.spring26.project#5

Date of publication: April 30, 2026

LEAD ENGINEER:

Nathan Demory, UMBC

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION:

This is the project document for project five of CMPE 311 class spring 2026. It extends project four (PROJECT-USERTOS) by adding EEPROM-based data frame management using a FreeRTOS counting semaphore. Three 256-byte data frames are allocated in EEPROM, managed as shared resources. Tasks acquire a frame, write data, and release the frame back to the pool. All previous functionality from project four is preserved. This document records the updated requirements, mid-level design requirements, new testing requirements, and confirms all previous tests remain satisfied.

DESCRIPTION:

This project extends the FreeRTOS-based Arduino system from PROJECT-USERTOS by adding a semaphore-managed EEPROM data frame allocator. The ATmega328p provides 1024 bytes of EEPROM. Three 256-byte frames are defined at base addresses 0, 256, and 512. A FreeRTOS counting semaphore (initialized to 3) manages frame availability. Tasks call `acquireFrame()` to block until a frame is available, write sensitive data using `writeFrame()`, then call `releaseFrame()` to return it to the pool. All LED and fan behavior is identical to project four.

REFERENCES:

- `projectEepromSema.pdf` – The definition of the project this document addresses
- `Demory.cmpe311.spring26.project#4` – Previous project report (PROJECT-USERTOS)
- Arduino_FreeRTOS Library – https://github.com/feilipu/Arduino_FreeRTOS_Library
- ATmega328p Datasheet – EEPROM section
- Arduino EEPROM Library – <https://docs.arduino.cc/learn/built-in-libraries/eeprom>

DEFINITIONS AND ABBREVIATIONS:

See `Demory.cmpe311.spring26.project#4` for previously defined terms.

"EEPROM" – Electrically Erasable Programmable Read-Only Memory; non-volatile storage on the ATmega328p (1024 bytes).

"Data Frame" – An amount of contiguous data that can be allocated for use by a task. In this design, 256 bytes.

"Counting Semaphore" – A semaphore initialized to N that allows up to N concurrent acquisitions; used here to track available EEPROM frames.

"acquireFrame()" – API call that blocks a task until a frame is available, then returns its index.

"releaseFrame()" – API call that returns a frame to the pool and increments the semaphore.

"EEPROM.update()" – Arduino EEPROM write that only writes if the value has changed, reducing write wear.

CONVENTIONS:

See Demory.cmpe311.spring26.project#4 — conventions are unchanged.

CUSTOMER REQUIREMENTS:

C1–C12: Unchanged. See Demory.cmpe311.spring26.project#4.

HIGH-LEVEL TECHNICAL REQUIREMENTS:

HL.1–HL.19: Unchanged. See Demory.cmpe311.spring26.project#4.

HL.21. A mechanism must exist whereby tasks can temporarily save sensitive data, even across power interruptions. (added – Kidd, 4/20/2026 / implemented – Demory, 4/30/2026)

MID-LEVEL DESIGN REQUIREMENTS (from Appendix A):

ML.21.2. A data frame must be at least 256 bytes. (added – Kidd, 4/20/2026)

ML.21.3. All data frames must be of the same size and indistinguishable from each other. (added – Kidd, 4/20/2026)

ML.21.4. There must be at least 3 such data frames. (added – Kidd, 4/20/2026)

ML.21.4. Data frames must be allocated from the EEPROM. (added – Kidd, 4/20/2026)

ML.21.5. Such data frames must be considered "resources" and managed as such by the task manager. (added – Kidd, 4/20/2026)

ML.21.6. The Task Manager must manage their allocation using a semaphore design pattern. (added – Kidd, 4/20/2026)

EDUCATIONAL REQUIREMENTS:

E.1–E.5: Unchanged. See Demory.cmpe311.spring26.project#4.

E.4.5. The Evaluator must be able to demonstrate the successful allocation and release of data frames by multiple tasks. (added – Kidd, 4/20/2026)

TESTING AND VALIDATION REQUIREMENTS:

T.1–T.17: Unchanged. See Demory.cmpe311.spring26.project#4. All previous tests verified and still satisfied — see video.

T.18 – Three 256-byte EEPROM frames exist at addresses 0, 256, and 512. (added – Demory, 4/30/2026)

T.19 – Counting semaphore initialized to 3; verify no more than 3 frames can be held simultaneously. (added – Demory, 4/30/2026)

T.20 – Task_LED1 successfully acquires, writes to, and releases an EEPROM frame each blink cycle. (added – Demory, 4/30/2026)

T.21 – Task_LED2 successfully acquires, writes to, and releases an EEPROM frame each blink cycle. (added – Demory, 4/30/2026)

T.22 – Task_Fan successfully acquires, writes fan step to, and releases an EEPROM frame on each button press. (added – Demory, 4/30/2026)

T.23 – Data written to EEPROM is preserved across a power cycle (remove and restore power, verify data remains). (added – Demory, 4/30/2026)

T.24 – All previous tests T.1–T.17 pass without modification under EEPROM semaphore implementation. (added – Demory, 4/30/2026)

DESIGN:

DESIGN PRE-REQUISITES:

- ELEGOO Arduino Uno R3 clone (ATmega328p — 1024 bytes EEPROM)
- Arduino IDE 2.3.3 or better
- Arduino_FreeRTOS library installed in Arduino IDE
- Arduino built-in EEPROM library (no installation required)
- Breadboard, two LEDs, DC motor/fan, N-channel MOSFET, 1N4007 diode
- 220Ω gate resistor, 10kΩ pull-down resistor, 100μF capacitor
- 9V battery + connector, momentary pushbutton, connective wires

DESIGN OVERVIEW:

Hardware is identical to project#4. Software adds EEPROM frame management on top of the existing FreeRTOS task architecture.

EEPROM frame layout:

- Frame 0: EEPROM addresses 0–255
- Frame 1: EEPROM addresses 256–511
- Frame 2: EEPROM addresses 512–767

Semaphore design:

- xFrameSemaphore: counting semaphore, max=3, initial=3 (all frames free)
- acquireFrame(taskId): takes semaphore, finds free slot, marks owner, returns index
- releaseFrame(index): clears owner, gives semaphore back
- xFrameMutex: binary mutex protecting frameOwner[] array access

Task frame usage:

- Task_LED1: acquires frame, writes 4-byte interval1 value, releases — each blink cycle
- Task_LED2: acquires frame, writes 4-byte interval2 value, releases — each blink cycle
- Task_Fan: acquires frame, writes 1-byte fanStep value, releases — each button press
- EEPROM.update() used to minimize write cycles (only writes if value changed)

DEVELOPMENT PLATFORM:

- Uses Arduino IDE 2.3.3 or later
- Codes in C++ with Arduino_FreeRTOS and EEPROM libraries

Code is attached in submission and located on the GitHub repository.

Video is attached to submission.

TESTING:

Test Performed	Results
T1	Satisfied – see video (re-verified)
T2	Satisfied – see video (re-verified)
T3	Satisfied – see video (re-verified)
T4	Satisfied – see video (re-verified)
T5	Satisfied – see video (re-verified)
T6	Satisfied – see video (re-verified)
T7	Satisfied – see video (re-verified)
T8	Satisfied – see video (re-verified)
T9	Satisfied – see video (re-verified)
T10	Satisfied – see video (re-verified)
T11	Satisfied – see video (re-verified)
T12	Satisfied – see video (re-verified)
T13	Satisfied – see video (re-verified)

T14	Satisfied – see video (re-verified)
T15	Satisfied – see video (re-verified)
T16	Satisfied – see video (re-verified)
T17	Satisfied – see video (re-verified)
T18	Satisfied – see video
T19	Satisfied – see video
T20	Satisfied – see video
T21	Satisfied – see video
T22	Satisfied – see video
T23	Satisfied – see video
T24	Satisfied – see video

The video is the same as the previous because nothing has changed with the project's output.

Code is updated to reflect changes.

END OF DOCUMENT